

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**  
**ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ**  
Кафедра програмної інженерії

**ЗВІТ**  
з лабораторної роботи № 3  
**«Модульне тестування програмного коду (Unit Testing)»**  
з дисципліни «Основи програмної інженерії»

Виконав: студент групи ПЗПІ-25-4  
Купрін Матвій Володимирович  
Перевірив:  
В. О. Гребенюк

Харків 2026

### 3.1 Мета роботи

Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів — еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %.

### 3.2 Посилання

Посилання на Git-репозиторій з кодом модуля та тестами:

<https://github.com/mk1234717/MenuQR/tree/KuprinMatvii/Unit%20Testing>

Посилання на uml модель з лр2:

<https://github.com/mk1234717/MenuQR/blob/KuprinMatvii/class%20diagram/uml-diagram.png>

Посилання на обрану мову програмування з пз1:

<https://github.com/mk1234717/MenuQR/blob/main/SRS.pdf>

### 3.3 Код модуля

Для написання коду був використаний DeepSeek, промпт:

You are a senior software engineer. I need to implement a Python module for unit testing lab work based on the following UML class diagram (from a restaurant QR menu system). The class is MenuItem with attributes: id: UUID, name: String, price: float, imageUrl: String. Add additional fields if needed for business logic.

Requirements:

- Implement at least 3 non-trivial methods (getters/setters don't count). Each method must contain:
  - conditional logic (if/else, switch) or loops
  - exception handling (raise ValueError, RuntimeError, etc.)
  - meaningful business rules (discounts, VAT, validation, etc.)
- Use Python with proper type hints.
- Use Decimal for price calculations (avoid float precision issues).
- Provide full class code with docstrings.
- Include helper classes if needed (SeasonalDiscount, ValidationResult).
- Do NOT include any file I/O, database, or external API calls – pure logic for unit testing.

The class must be easy to test with pytest. Show me the final code.

Код: import uuid

```
from typing import List, Optional, Callable

from decimal import Decimal, ROUND_HALF_UP
```

```
class SeasonalDiscount:
```

```
    """ДТО для сезонних/акційних знижок."""
```

```
    def __init__(self, name: str, percent: float, applies_to: Callable[['MenuItem'], bool]):
```

```
        self.name = name
```

```
        self.percent = percent # 0-100
```

```
        self.applies_to = applies_to
```

```
class ValidationResult:
```

```
    """Результат валідації."""
```

```
    def __init__(self, is_valid: bool = True, errors: List[str] = None):
```

```
        self.is_valid = is_valid
```

```
        self.errors = errors or []
```

```
class MenuItem:
```

```
    """
```

```
    Страва в меню закладу.
```

```
    """
```

```
    def __init__(self, name: str, price: float, image_url: str = None,
```

```
                 is_available: bool = True, special_discount_percent: Optional[float] = None):
```

```
        """
```

```
        Конструктор з валідацією (обробка виняткових ситуацій).
```

```
        """
```

```
        if not name or not name.strip():
```

```
            raise ValueError("Назва страви не може бути порожньою.")
```

```

if price < 0:

    raise ValueError("Ціна не може бути від'ємною.")

if price > 1_000_000:

    raise ValueError("Ціна перевищує максимально допустиму (1 000 000).")

if special_discount_percent is not None and (special_discount_percent < 0 or
special_discount_percent > 100):

    raise ValueError("Знижка має бути в діапазоні 0–100%.")


self.id = uuid.uuid4()

self.name = name.strip()

self.price = Decimal(str(price))

self.image_url = image_url or ""

self.is_available = is_available

self.special_discount_percent = Decimal(str(special_discount_percent)) if
special_discount_percent is not None else None


# -----

# Метод №1 (нетривіальний): Розрахунок фінальної ціни зі знижками

# -----

def calculate_final_price(self, active_discounts: Optional[List[SeasonalDiscount]] = None) ->
float:

    """

    Повертає ціну після застосування спеціальної знижки та всіх активних сезонних
    акцій.

    - Якщо страва недоступна -> виняток.

    - Цикл по active_discounts.

    - Умовні конструкції для перевірки знижок та захисту від від'ємної ціни.

    """

    if not self.is_available:

        raise RuntimeError(f"Страва '{self.name}' недоступна для замовлення.")

```

```
current_price = self.price
```

```
# 1. Спеціальна знижка на страву
```

```
if self.special_discount_percent is not None:
```

```
    current_price -= current_price * (self.special_discount_percent / 100)
```

```
# 2. Сезонні / накопичувальні знижки (цикл)
```

```
if active_discounts:
```

```
    for discount in active_discounts:
```

```
        if discount.applies_to(self):
```

```
            current_price -= current_price * (Decimal(str(discount.percent)) / 100)
```

```
# Захист від від'ємної ціни після всіх знижок
```

```
if current_price < 0:
```

```
    current_price = Decimal('0')
```

```
# Округлення до 2 знаків (грошовий стандарт)
```

```
return float(current_price.quantize(Decimal('0.01'), rounding=ROUND_HALF_UP))
```

```
# -----
```

```
# Метод №2 (нетривіальний): Розрахунок ціни з ПДВ (умовна логіка + ставки)
```

```
# -----
```

```
def calculate_price_with_vat(self, vat_rate: str) -> float:
```

```
    """
```

```
    Повертає ціну страви (без знижок) із доданим ПДВ.
```

```
    Підтримує ставки: 'zero', 'reduced_7', 'standard_20'.
```

```
    """
```

```

vat_rates = {
    'zero': 1.00,
    'reduced_7': 1.07,
    'standard_20': 1.20
}

if vat_rate not in vat_rates:
    raise ValueError(f'Невідома ставка ПДВ: {vat_rate}. Допустимі: zero, reduced_7,
standard_20')

multiplier = Decimal(str(vat_rates[vat_rate]))
price_with_vat = self.price * multiplier
return float(price_with_vat.quantize(Decimal('0.01'), rounding=ROUND_HALF_UP))

# -----
# Метод №3 (нетривіальний): Комплексна валідація перед публікацією
# -----

def validate_for_publish(self) -> ValidationResult:
    """
    Перевіряє всі поля страви: назва, ціна, знижка, URL зображення.
    Повертає ValidationResult зі списком помилок.
    """

    errors = []

    # Перевірка назви
    if not self.name or len(self.name.strip()) < 2:
        errors.append("Назва повинна містити хоча б 2 символи.")
    elif len(self.name) > 100:
        errors.append("Назва не може перевищувати 100 символів.")

```

```

# Перевірка ціни

if self.price < 0:

    errors.append("Ціна не може бути від'ємною.")

elif self.price == 0:

    errors.append("Ціна не може дорівнювати нулю (безкоштовні страви не підтримуються).")

elif self.price > 10000:

    errors.append("Ціна занадто висока (> 10000).")


# Перевірка спеціальної знижки

if self.special_discount_percent is not None:

    if self.special_discount_percent < 0 or self.special_discount_percent > 100:

        errors.append("Знижка має бути від 0% до 100%.")

    elif self.special_discount_percent == 100 and self.price > 0:

        errors.append("Страва не може бути повністю безкоштовною через знижку 100%.")


# Перевірка URL зображення (якщо вказано)

if self.image_url:

    import re

    url_pattern = re.compile(r'^https?:/\S+$')

    if not url_pattern.match(self.image_url):

        errors.append("Посилання на зображення має бути коректним URL (http:// або https://).")


return ValidationResult(is_valid=len(errors) == 0, errors=errors)


# -----

# Допоміжні методи (зміна стану) – не є основними для тестування логіки

```

# -----

```
def set_availability(self, is_available: bool) -> None:
```

```
    self.is_available = is_available
```

```
def set_special_discount(self, discount_percent: Optional[float]) -> None:
```

```
    if discount_percent is not None and (discount_percent < 0 or discount_percent > 100):
```

```
        raise ValueError("Знижка має бути в межах 0–100.")
```

```
    self.special_discount_percent = Decimal(str(discount_percent)) if discount_percent is not  
    None else None
```

### 3.4 Таблиця проєктування тестів

| №                              | Тест-кейс                            | Вхідні дані                                                             | Очікуваний результат   | Техніка (EP/BVA)          | Статус (pass/fail) |
|--------------------------------|--------------------------------------|-------------------------------------------------------------------------|------------------------|---------------------------|--------------------|
| Метод calculate_final_price    |                                      |                                                                         |                        |                           |                    |
| 1                              | Страва доступна, без знижок          | price=100.0,<br>is_available=True,<br>active_discounts=None             | 100.0                  | EP (доступна)             | pass               |
| 2                              | Страва недоступна                    | is_available=False                                                      | RuntimeError           | EP (недоступна → виняток) | pass               |
| 3                              | Спеціальна знижка 25%                | price=200.0,<br>special_discount=25,<br>без сезонних                    | 150.0                  | EP (знижка в діапазоні)   | pass               |
| 4                              | Граничні значення знижки: 0% та 100% | price=100,<br>special_discount=100 → 0.0;<br>special_discount=0 → 100.0 | 0.0 та 100.0           | BVA (межі 0 та 100)       | pass               |
| 5                              | Цикл: кілька сезонних знижок         | price=100,<br>active_discounts=[10%, 20%]                               | 100 * 0.9 * 0.8 = 72.0 | EP (цикл, комбінація)     | pass               |
| Метод calculate_price_with_vat |                                      |                                                                         |                        |                           |                    |
| 6                              | Ставка ПДВ 20% (standard)            | price=100.0,<br>vat_rate='standard_20'                                  | 120.00                 | EP (допустима ставка)     | pass               |
| 7                              | Ставка 7% (reduced)                  | price=100.0,<br>vat_rate='reduced_7'                                    | 107.00                 | EP (допустима ставка)     | pass               |
| 8                              | Невідома ставка                      | vat_rate='unknown'                                                      | ValueError             | EP (недопустиме значення) | pass               |
| 9                              | Граничні значення ціни: 0 та 0.01    | price=0,<br>vat_rate='standard_20' → 0.0; price=0.01 → 0.01 (після      | 0.0 та 0.01            | BVA (межі ціни)           | pass               |

|                                         |                           |                                                                                                                        |                            |                          |      |
|-----------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------|--------------------------|------|
|                                         |                           | округлення)                                                                                                            |                            |                          |      |
| Метод <code>validate_for_publish</code> |                           |                                                                                                                        |                            |                          |      |
| 10                                      | Валідна страва            | <code>name="Піца",<br/>price=150,<br/>image_url="https://..."</code>                                                   | <code>is_valid=True</code> | ЕР (все коректно)        | pass |
| 11                                      | Невалідна назва (границі) | довжина назви 1 символ → помилка; 101 символ → помилка; 2 символи → ОК                                                 | помилки на межах           | BVA (1,2,100,101)        | pass |
| 12                                      | Невалідна ціна (границі)  | <code>price=0</code> → помилка; <code>price=10000.01</code> → помилка; <code>price=-0.01</code> → виняток конструктора | помилки або виняток        | BVA (0, 10000, від'ємна) | pass |

### 3.5 Код модульних тестів

```
import pytest
```

```
from menu_item import MenuItem, SeasonalDiscount, ValidationResult
```

```
# -----
```

```
# Тести для методу calculate_final_price
```

```
# -----
```

```
def test_calculate_final_price_available_no_discount():
```

```
    """ЕР: доступна страва без знижок -> повертає базову ціну"""
```

```
    # Arrange
```

```
    item = MenuItem(name="Борщ", price=100.0, is_available=True)
```

```
    # Act
```

```
    result = item.calculate_final_price(active_discounts=None)
```

```
    # Assert
```

```
    assert result == 100.0
```

```
def test_calculate_final_price_unavailable():
```

```
    """ЕР: недоступна страва -> виняток RuntimeError"""
```

```
# Arrange
```

```
item = MenuItem(name="Борщ", price=100.0, is_available=False)
```

```
# Act & Assert
```

```
with pytest.raises(RuntimeError, match="Страва 'Борщ' недоступна"):
```

```
    item.calculate_final_price()
```

```
def test_calculate_final_price_special_discount_25():
```

```
    """EP: спеціальна знижка 25% -> ціна зменшується на 25%"""
```

```
# Arrange
```

```
item = MenuItem(name="Борщ", price=200.0, special_discount_percent=25.0)
```

```
# Act
```

```
result = item.calculate_final_price()
```

```
# Assert
```

```
assert result == 150.0
```

```
def test_calculate_final_price_special_discount_boundaries():
```

```
    """BVA: перевірка граничних значень знижки 0% та 100%"""
```

```
# Arrange
```

```
item0 = MenuItem(name="Страва0", price=100.0, special_discount_percent=0)
```

```
item100 = MenuItem(name="Страва100", price=100.0, special_discount_percent=100)
```

```
# Act
```

```
result0 = item0.calculate_final_price()
```

```
result100 = item100.calculate_final_price()
```

```
# Assert
```

```
assert result0 == 100.0
```

```
assert result100 == 0.0
```

```
def test_calculate_final_price_multiple_seasonal_discounts():
```

```

"""ЕР: цикл – кілька сезонних знижок, що застосовуються"""

# Arrange

item = MenuItem(name="Страва", price=100.0)

disc1 = SeasonalDiscount("Sale1", 10, lambda m: True)

disc2 = SeasonalDiscount("Sale2", 20, lambda m: True)

# Act

result = item.calculate_final_price(active_discounts=[disc1, disc2])

# Assert

# 100 * 0.9 * 0.8 = 72.0

assert result == 72.0


# -----

# Тести для методу calculate_price_with_vat

# -----


def test_calculate_price_with_vat_standard():

    """ЕР: ставка ПДВ 20% -> ціна збільшується на 20%"""

    # Arrange

    item = MenuItem(name="Страва", price=100.0)

    # Act

    result = item.calculate_price_with_vat("standard_20")

    # Assert

    assert result == 120.00


def test_calculate_price_with_vat_reduced():

    """ЕР: ставка ПДВ 7% -> ціна збільшується на 7%"""

    # Arrange

    item = MenuItem(name="Страва", price=100.0)

```

```

# Act

result = item.calculate_price_with_vat("reduced_7")

# Assert

assert result == 107.00


def test_calculate_price_with_vat_invalid_rate():

    """EP: невідома ставка ПДВ -> виняток ValueError"""

    # Arrange

    item = MenuItem(name="Страва", price=100.0)

    # Act & Assert

    with pytest.raises(ValueError, match="Невідома ставка ПДВ"):

        item.calculate_price_with_vat("unknown")


def test_calculate_price_with_vat_price_boundaries():

    """BVA: перевірка граничних значень ціни (0 та 0.01)"""

    # Arrange

    item_zero = MenuItem(name="Нуль", price=0.0)

    item_small = MenuItem(name="Мала", price=0.01)

    # Act

    result_zero = item_zero.calculate_price_with_vat("standard_20")

    result_small = item_small.calculate_price_with_vat("standard_20")

    # Assert

    assert result_zero == 0.0

    assert result_small == 0.01 # 0.01 * 1.20 = 0.012 -> округлення до 0.01


# -----

# Тести для методу validate_for_publish

# -----

```

```

def test_validate_for_publish_valid():

    """EP: всі поля коректні -> is_valid = True"""

    # Arrange

    item = MenuItem(

        name="Піца",

        price=150.0,

        image_url="https://example.com/pizza.jpg"

    )

    # Act

    result = item.validate_for_publish()

    # Assert

    assert result.is_valid is True

    assert result.errors == []


def test_validate_for_publish_name_boundaries():

    """BVA: перевірка границь довжини назви (1, 2, 100, 101 символів)"""

    # Тест для назви з 1 символу (невалідно)

    item1 = MenuItem(name="A", price=100.0)

    result1 = item1.validate_for_publish()

    assert result1.is_valid is False

    assert any("хоча б 2 символи" in err for err in result1.errors)


    # Тест для назви з 2 символів (валідно)

    item2 = MenuItem(name="AB", price=100.0)

    result2 = item2.validate_for_publish()

    # Інші поля валідні? Ціна 100 > 0, знижки немає – має бути valid

    assert result2.is_valid is True

```

```

# Тест для назви зі 100 символів (валідно)

name100 = "X" * 100

item100 = MenuItem(name=name100, price=100.0)

result100 = item100.validate_for_publish()

assert result100.is_valid is True


# Тест для назви з 101 символу (невалідно)

name101 = "X" * 101

item101 = MenuItem(name=name101, price=100.0)

result101 = item101.validate_for_publish()

assert result101.is_valid is False

assert any("не може перевищувати 100" in err for err in result101.errors)


def test_validate_for_publish_price_boundaries():

    """BVA: перевірка границь ціни (0, 10000.01, від'ємна)"""

    # Ціна = 0 (невалідно через конструктор? Конструктор price=0 дозволяє, але валідація забороняє)

    item_zero = MenuItem(name="Десерт", price=0.0)

    result_zero = item_zero.validate_for_publish()

    assert result_zero.is_valid is False

    assert any("не може дорівнювати нулю" in err for err in result_zero.errors)


    # Ціна > 10000 (невалідно)

    item_high = MenuItem(name="Люкс", price=10000.01)

    result_high = item_high.validate_for_publish()

    assert result_high.is_valid is False

    assert any("Ціна занадто висока" in err for err in result_high.errors)

```

```
# Від'ємна ціна – перевіряємо, що конструктор кидає ValueError (не доходимо до
validate_for_publish)
```

```
with pytest.raises(ValueError, match="Ціна не може бути від'ємною"):
```

```
MenuItem(name="Помилка", price=-0.01)
```

**Код для запуску тесту:** `pytest test_menu_item.py --cov=menu_item --cov-report=term --cov-report=html`

Coverage report: 82%

Files Functions Classes

coverage.py v7.14.0, created at 2026-05-13 13:07 +0300

| File ▲       | statements | missing   | excluded | coverage   |
|--------------|------------|-----------|----------|------------|
| menu_item.py | 77         | 14        | 0        | 82%        |
| <b>Total</b> | <b>77</b>  | <b>14</b> | <b>0</b> | <b>82%</b> |

coverage.py v7.14.0, created at 2026-05-13 13:07 +0300

Рис.1 – Звіт покриття, line coverage = 82%

## Додаткові тести для покращення покриття

```
# -----
```

```
# Додаткові тести для досягнення 100% покриття (ітеративний цикл)
```

```
# -----
```

```
def test_calculate_final_price_negative_protection():
```

```
    """Перевірка захисної логіки: якщо сумарна знижка >100% -> ціна стає 0"""
```

```
    # Arrange: створюємо страву з ціною 100, спеціальною знижкою 80%
```

```
    item = MenuItem(name="Тест", price=100.0, special_discount_percent=80.0)
```

```
    # Сезонна знижка 50% (разом 130% знижки)
```

```
    disc = SeasonalDiscount("Super", 50, lambda m: True)
```

```
    # Act
```

```
    result = item.calculate_final_price(active_discounts=[disc])
```

```
    # Assert: ціна має стати 0, а не від'ємною
```

```
    assert result == 0.0
```

```
def test_calculate_final_price_rounding():
```

```

"""BVA: перевірка округлення до 2 знаків (наприклад, 0.005 -> 0.01)"""

# Arrange

item = MenuItem(name="Кава", price=0.03, special_discount_percent=33.33)

# 0.03 * (1 - 0.3333) = 0.03 * 0.6667 = 0.020001 -> округлення до 0.02

# Але точніше: Decimal('0.03') * (Decimal('66.67')/100) = ?

# Спростимо: price=0.07, знижка 50% -> 0.035 -> округлення до 0.04

item2 = MenuItem(name="Чай", price=0.07, special_discount_percent=50.0)

# Act

result = item2.calculate_final_price()

# Assert: 0.07 * 0.5 = 0.035 -> округлюється до 0.04 (bankers rounding? Python
ROUND_HALF_UP)

assert result == 0.04


def test_validate_for_publish_discount_100_percent():

    """EP: спеціальна знижка 100% при ціні >0 -> невалідно"""

    # Arrange

    item = MenuItem(name="Безкоштовно?", price=50.0, special_discount_percent=100.0)

    # Act

    result = item.validate_for_publish()

    # Assert

    assert result.is_valid is False

    assert any("безкоштовною" in err for err in result.errors)


def test_validate_for_publish_invalid_image_url():

    """EP: некоректний URL зображення -> помилка"""

    # Arrange

    item = MenuItem(

        name="Страва",

        price=100.0,

```

```

        image_url="ftp://not-http-url.com"
    )

    # Act

    result = item.validate_for_publish()

    # Assert

    assert result.is_valid is False

    assert any("коректним URL" in err for err in result.errors)


def test_constructor_price_exceeds_maximum():
    """BVA: ціна більше 1_000_000 -> виняток"""

    with pytest.raises(ValueError, match="перевищує максимально допустиму"):
        MenuItem(name="Дорого", price=1_000_001.0)


def test_constructor_price_at_maximum():
    """BVA: ціна дорівнює 1_000_000 -> допустимо"""

    item = MenuItem(name="Максимум", price=1_000_000.0)

    assert item.price == 1_000_000.0


def test_set_availability():
    """Позитивний тест для допоміжного методу set_availability (покриття рядків)"""

    item = MenuItem(name="Страва", price=100.0)

    assert item.is_available is True # за замовчуванням

    item.set_availability(False)

    assert item.is_available is False

    item.set_availability(True)

    assert item.is_available is True


def test_set_special_discount():

```

```

"""Позитивний та негативний тести для set_special_discount"""

item = MenuItem(name="Страва", price=100.0)

item.set_special_discount(25.0)

assert item.special_discount_percent == 25.0


item.set_special_discount(None)

assert item.special_discount_percent is None


# Негативний: знижка >100

with pytest.raises(ValueError, match="Знижка має бути в межах 0–100"):

    item.set_special_discount(150.0)


with pytest.raises(ValueError, match="Знижка має бути в межах 0–100"):

    item.set_special_discount(-10.0)

```

| Coverage report: 94%                                     |            |          |          |            |
|----------------------------------------------------------|------------|----------|----------|------------|
| <div>Files</div> <div>Functions</div> <div>Classes</div> |            |          |          |            |
| coverage.py v7.14.0, created at 2026-05-13 13:18 +0300   |            |          |          |            |
| File ▲                                                   | statements | missing  | excluded | coverage   |
| menu_item.py                                             | 77         | 5        | 0        | 94%        |
| <b>Total</b>                                             | <b>77</b>  | <b>5</b> | <b>0</b> | <b>94%</b> |
| coverage.py v7.14.0, created at 2026-05-13 13:18 +0300   |            |          |          |            |

Рис.2 – Звіт покриття, line coverage = 94%

$$\Delta = 94\% - 82\% = 12\%$$

Отже, вдалося покращити покриття на 12%.

## Висновки

Покриття коду: Початковий рівень line coverage становив 82% (14 непокритих рядків). Після додавання 8 тестів покриття підвищилося до 94% (залишилося 5 непокритих

рядків). Основні непокриті рядки стосувалися виняткових ситуацій конструктора (порожня назва, ціна >1 000 000, знижка поза 0–100) та деяких гілок валідації.

Проблеми: Деякі перевірки дублюються в конструкторі та в методі ``validate_for_publish``, через що окремі рядки ніколи не виконуються (наприклад, обробка від'ємної ціни в ``validate_for_publish``, оскільки конструктор блокує створення такого об'єкта).

Шляхи поліпшення: Винести спільну валідацію в окремий допоміжний метод, використовувати параметризовані тести для BVA, а для досягнення 100% покриття – застосувати техніки інкапсуляції (наприклад, тестування через відкриті методи з моками).